

DeepSeek关键RL算法GRPO，有人从头跑通了，贡献完整代码

机器之心 2025年03月02日 11:54 北京

选自GitHub

作者：Andriy Burkov

机器之心编译

GRPO（Group Relative Policy Optimization）是 DeepSeek-R1 成功的基础技术之一，我们之前也多次报道过该技术，比如《[DeepSeek 用的 GRPO 占用大量内存？有人给出了些破解方法](#)》。

简单来说，GRPO 算法丢弃了 critic model，放弃了价值函数近似，转而通过组内样本的相对比较来计算策略梯度，从而有效降低了训练的不稳定性，同时提高了学习效率。

既然 GRPO 如此有效，那么，你知道如何从头开始实现 GRPO 吗？

近日，AI 工程师和技术作家 Andriy Burkov 发布了一份「从头开始写 GRPO 代码」的教程，其中介绍了如何基于 Qwen2.5-1.5B-Instruct 模型构建一个使用 GRPO 的分布式强化学习流程。

不过，在我们深入这份教程之前，先简单介绍一下它的作者。Andriy Burkov 算得上是 AI 领域的一位著名科普作家，在加拿大拉瓦尔大学取得了计算机科学博士学位，还曾发表过两本颇受欢迎的 AI 主题著作：《100 页语言模型书》和《100 页机器学习书》；书中一步步详实地介绍了相关概念，并附带了简明的实现代码。



Andriy Burkov

@burkov

...

I just published a detailed tutorial on how to code GRPO (the reinforcement learning algorithm used to train DeepSeek R1) from scratch and how to finetune the Qwen-2.5-1.5B-Instruct model using GRPO to solve 90% of the problems from the GSM8K dataset of high-quality, linguistically diverse grade school math problems created by human problem writers.

The tutorial also shows how to use PyTorch's DataParallel to distribute the training across several GPUs:
[github.com/aburkov/theLMb...](#)

To reproduce the DataParallel code, you will need a node with at least 2 GPUs with 80GB of VRAM or more (8 GPUs recommended). If you purchased my The Hundred-Page Language Models Book, you can claim \$150 in free cloud GPU credits on [@LambdaAPI](#) by sending me proof of purchase at author@thelmbok.com. With these credits, you can run an 8xA100 node on Lambda. I adjusted the tutorial for running on this node.

Happy hacking!

由 Google 翻译自 英语

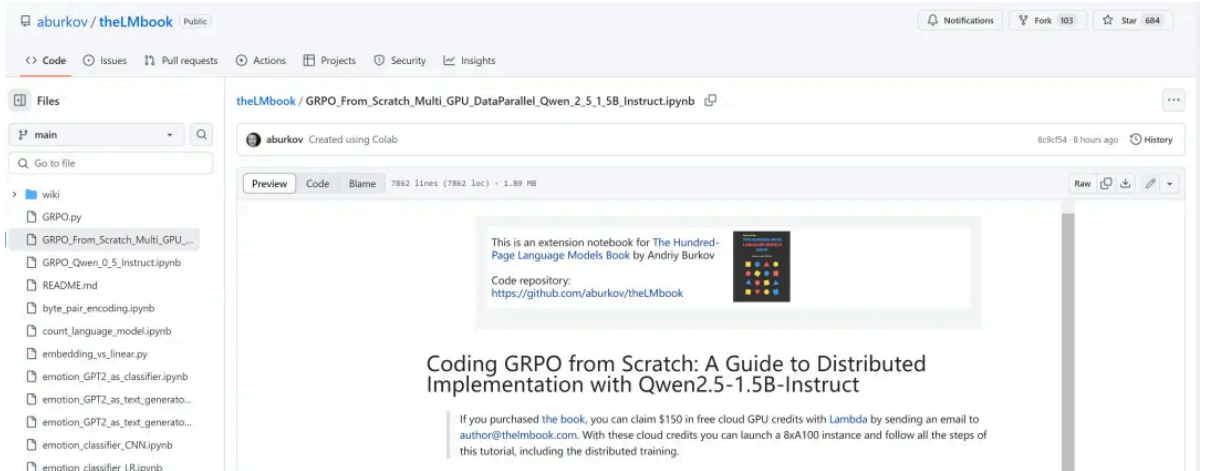
我刚刚发布了一个详细的教程，介绍如何从头开始编写 GRPO（用于训练 DeepSeek R1 的强化学习算法），以及如何使用 GRPO 微调 Qwen-2.5-1.5B-Instruct 模型，以解决 GSM8K 数据集中 90% 的问题，该数据集包含由人类问题作者创建的高质量、语言多样化的小学数学问题。

本教程还展示了如何使用 PyTorch 的 DataParallel 在多个 GPU 上分配训练：
[github.com/aburkov/theLMb...](#)

要重现 DataParallel 代码，您需要一个至少具有 2 个 GPU 的节点，并且 VRAM 至少为 80GB (8 GPU。如果您购买了 my 的《百页语言模型》一书，您可以通过向 author@thelmbok.com 发送购买证明，在 [@LambdaAPI](#) 上申请 150 美元的免费云 GPU 积分。有了这些积分，您可以在 Lambda 上运行 8xA100 节点。我调整了教程以在此节点上运行。

快乐黑客！

接下来我们就来看看这份 GRPO 从头实现教程吧。



教程地址：

https://github.com/aburkov/theLMbook/blob/main/GRPO_From_Scratch_Multi_GPU_DataParallel_Qwen_2_5_1_5B_Instruct.ipynb

从头编写 GRPO 代码

使用 Qwen2.5-1.5B-Instruct 的分布式实现

本教程将展示如何使用 GRPO 方法构建分布式强化学习（RL）流程，从而可以针对数学、逻辑和编程任务对语言模型进行微调。

首先需要明确，这些任务都存在一个唯一且正确的 `ground truth` 答案，可通过简单的字符串比较轻松加以验证。

GRPO 的发明者是 DeepSeek，最早是被用于微调 DeepSeek 的 R1 和 R1-Zero 模型 —— 它们可通过学习生成思维链（CoT）来更好地解决数学和逻辑问题。

本教程的目标是将通用语言模型 Qwen2.5-1.5B-Instruct 转换为数学问题求解器。我们将从头开始编写 GRPO 代码，然后将其与几个流行的库和工具集成起来，以实现分布式训练管道流程，包括：

- `PyTorch`：用于张量运算和分布式训练。
- `Hugging Face Transformers`：用于加载预训练的语言模型和 `tokenizer`。
- `FlashAttention2`：优化的注意力机制，有助于减少内存使用量并提高训练速度。
- `Weights & Biases (wandb)`：用于实验跟踪、可视化和模型版本控制。

本教程分为几个部分。首先是基本设置和导入，然后是数据格式化和答案提取、数据集准备、评估函数、奖励函数、训练设置和执行，最后加载和测试模型。此过程中，我们将从头实现 GRPO 算法。

Part 1：基础设置和导入

首先是安装并导入所有必要的模块。下面是导入库的一段代码截图。

```
!pip install tf-keras # for some reason, Hugging Face cannot work without it
!pip install flash-attn # FlashAttention2
!pip install wandb # Weights and Biases
!pip install 'accelerate>=0.26.0'
!pip install transformers # Hugging Face Transformers API
!pip install datasets # Hugging Face Datasets API

# Import necessary libraries
# Basic Python libraries for various operations
import random
import copy
import re
import os
import numpy as np
import wandb

# PyTorch and related libraries for deep learning
import torch
import torch.nn as nn
from torch.nn.utils.rnn import pad_sequence

# Hugging Face libraries for transformer models
from transformers import AutoModelForCausalLM, AutoTokenizer
from datasets import load_dataset
```

部分代码截图。完整代码块参见 [GitHub](#)。

运行上述代码（参考项目完整代码），可以执行以下任务：

- 设置随机种子：`set_random_seed` 函数通过为 Python 的随机模块、NumPy 和 PyTorch 设置种子，确保可复现性；
- 环境变量配置：设置 `WANDB_API_KEY` 和 `WANDB_PROJECT` 环境变量，以启用与 `Weights & Biases` 的实验跟踪；
- 导入必要的库，包括 `random`、`copy`、`re`、`torch` 等等。

Part 2：数据格式以及答案提取

接下来，项目定义了数据格式，以及模型如何从输出和数据集中提取答案段落。

为了确保模型输出格式一致，项目还定义了一个系统提示。该提示指示模型生成包含 `< reasoning >` 和 `< answer >` 标签的输出。这一步通过两个函数完成：

- `extract_answer_from_model_output`：此函数获取模型的输出文本，并提取 `< answer >` 标签内的内容；

- `extract_answer_from_dataset`：此函数从 GSM8K 数据集中提取预期答案，该数据集使用 “####” 分隔符来分隔答案：

```
def extract_answer_from_dataset(text):  
    """  
    Extracts the answer from the GSM8K dataset examples.  
  
    Args:  
        text (str): The dataset example text containing a question and answer.  
  
    Returns:  
        str or None: The extracted answer part after the '####' delimiter, or None if not found.  
  
    Explanation:  
        1. Checks if the text contains the '####' delimiter that separates question from answer.  
        2. If found, splits the text at this delimiter and returns the second part (the answer).  
        3. The answer is stripped of leading/trailing whitespace.  
        4. Returns None if no delimiter is present.  
    """  
  
    if "####" not in text:  
        return None  
    return text.split("####")[1].strip()
```

部分代码截图。完整代码块参见 [GitHub](#)。

Part 3：数据准备

该项目使用 GSM8K 数据集进行训练。项目使用了该数据集中的示例来训练模型，基于强化学习（RL）训练范式，让模型生成多个问题解答样本，之后作者将这些解答与 GSM8K 示例中的标准答案进行对比，如果匹配，就为 RL 算法（GRPO）提供高奖励，然后更新模型权重，以增加模型下次获得高奖励的可能性。

实验过程是这样的。首先从 Hugging Face 加载数据集，然后格式化每个示例，包括系统提示和用户提示。这段实现代码中还定义了两个辅助函数：`prepare_dataset` 以及 `build_prompt`。

```
def prepare_dataset(split="train"):  
    """  
    Load and prepare the GSM8K dataset for training with string prompts.  
  
    Args:  
        split (str): The dataset split to load ("train" or "test"). Defaults to "train".  
  
    Returns:  
        list: A list of formatted examples, each containing a prompt string and answer.  
  
    Explanation:  
        1. Loads the GSM8K dataset from the Hugging Face datasets hub.  
        2. For each example in the dataset:  
            - Creates a list of messages with system prompt and the question.  
            - Converts this list into a single string prompt using build_prompt().  
            - Extracts the answer from the dataset example.  
            - Creates a formatted example dictionary with prompt and answer.  
        3. Returns the list of formatted examples ready for model training or evaluation.  
    """  
  
    data = load_dataset('openai/gsm8k', 'main')[split]  
    formatted_data = []  
    for example in data:  
        # Convert list of messages to a single string prompt.  
        prompt_str = build_prompt([  
            {"role": "system", "content": SYSTEM_PROMPT},  
            {"role": "user", "content": example["question"]}  
        ])  
        formatted_example = {  
            "prompt": prompt_str, # Now a string rather than a list.  
            "answer": extract_answer_from_dataset(example["answer"])  
        }  
        formatted_data.append(formatted_example)  
    return formatted_data
```

部分代码截图。完整代码块参见 [GitHub](#)。

Part 4：评估函数

评估对于跟踪模型的进展至关重要。因此作者定义了一些函数，从而可以在一组示例上对模型进行评估。该项目的评估函数执行以下任务：

- `token` 化提示并生成响应：模型的输出是在 `token` 化提示的基础上生成的。
- 提取预测答案：从生成的响应中提取答案。
- 将预测答案与预期答案进行比较：这种比较是通过精确匹配以及数值等价检查来完成的。

在这段代码中，两个辅助函数 `_extract_last_number` 和 `_extract_single_number` 被用来从文本中提取数字。评估函数 `evaluate_model` 使用这些辅助函数来确定预测答案是否正确：

```
def evaluate_model(model, tokenizer, eval_examples, device):
    """
    Evaluates the model on a set of examples and prints detailed results.

    Args:
        model: The language model to evaluate.
        tokenizer: The tokenizer for encoding inputs and decoding outputs.
        eval_examples (list): List of evaluation examples, each containing "prompt" and "answer".
        device: The device (CPU or GPU) to run evaluation on.

    Returns:
        float: The accuracy percentage (correct predictions / total examples * 100).

    Explanation:
        1. Sets the model to evaluation mode.
        2. For each example in the evaluation set:
            - Encodes the prompt and generates a response using the model.
            - Extracts the predicted answer from the generated response.
            - Compares the predicted answer with the expected answer using multiple methods:
                a. Exact string matching
                b. Single number extraction and comparison
                c. Last number extraction and comparison
            - Prints detailed information about each example.
        3. Calculates and returns the overall accuracy.
        4. Returns the model to training mode.
    """
    model.eval()
    correct = 0
    total = len(eval_examples)
    print("\n" + "="*50)
    print("EVALUATION ON", total, "EXAMPLES")
    print("="*50)
```

部分代码截图。完整代码块参见 [GitHub](#)。

Part 5：奖励函数

在强化学习中，奖励函数是必不可缺的，作者定义了两个奖励函数：

correctness_reward：这个函数根据生成的答案是否正确来分配奖励。采用两种方式：精确的字符串匹配和数值等价检查，将模型输出的答案与预期答案进行比较。完全匹配会获得更高的奖励（2.0），而基于数值等价的匹配会获得较小的奖励（1.5）。

format_reward：这个函数鼓励模型遵循所需的类似 XML 的输出格式。它为生成文本中存在 `<reasoning>`、`</reasoning>`、`<answer >` 和 `</answer >` 标签提供小额奖励。

```
def correctness_reward(prompts, completions, answer, **kwargs):
    """
    Assigns a reward based on the correctness of the model's answer.

    Args:
        prompts (list): List of input prompts.
        completions (list): List of model completions, each containing content.
        answer (list): List of expected answers.
        **kwargs: Additional keyword arguments.

    Returns:
        list: List of numerical rewards for each completion.

    Explanation:
        1. Extracts the content from each completion.
        2. Extracts the answer portion from each response using extract_answer_from_model_output.
        3. Assigns rewards based on matching criteria:
            - 2.0 points for an exact match
            - 1.5 points for numeric equivalence (when values match but format differs)
            - 0.0 points for incorrect answers
        4. Tracks completion lengths for analysis.
    """
    responses = [completion[0]['content'] for completion in completions]
    extracted = [extract_answer_from_model_output(r) for r in responses]
    rewards = []
    for r, a in zip(extracted, answer):
        if r == a: # Exact match case
            rewards.append(2.0)
        else:
            # Try numeric equivalence
            r_num = extract_single_number(str(r))
            a_num = extract_single_number(str(a))
            if r_num is not None and a_num is not None and r_num == a_num:
                rewards.append(1.5)
            else:
                rewards.append(0.0)
    # Log completion lengths
    completion_lengths = [len(response.split()) for response in responses]
    return rewards
```

部分代码截图。完整代码块参见 *GitHub*。

Part 6：从头开始实现 DataParallel GRPO

这一节，我们将从头实现 GRPO 算法的所有构建模块。首先，这里假设运行代码的机器至少有 2 台 GPU。为此，这里要使用 PyTorch 的 DataParallel API 来将策略模型放在多个 GPU 核心上，每个 GPU 核心都有该模型的一个副本。然后将批量数据分散在这些 GPU 核心上完成处理。

```
def selective_log_softmax(logits, input_ids):  
    """  
    Computes log probabilities for specific tokens in the vocabulary.  
  
    Args:  
        logits (torch.Tensor): The raw logits output from the model.  
        input_ids (torch.Tensor): The token IDs for which we want the log probabilities.  
  
    Returns:  
        torch.Tensor: Log probabilities of the selected tokens.  
  
    Explanation:  
        1. Applies log softmax to convert logits to log probabilities over the vocabulary.  
        2. Uses gather to extract only the log probabilities corresponding to the input_ids.  
        3. Removes the extra dimension to match the original shape of input_ids.  
    """  
    log_probs = nn.functional.log_softmax(logits, dim=-1)  
    return log_probs.gather(dim=-1, index=input_ids.unsqueeze(-1)).squeeze(-1)
```

部分代码截图。完整代码块参见 *GitHub*。

Part 7：训练设置和执行

这一节，我们将所有组件组合在一起，完成设置并开始训练。

首先，加载预训练的模型和 tokenizer，准备评估数据，然后使用上面从头实现的 train_with_grpo 进行强化学习微调。

关键步骤包括：

- 模型和 tokenizer 初始化：使用优化设置（使用 torch.bfloat16 和 FlashAttention2）加载模型 Qwen/Qwen2.5-1.5B-Instruct。tokenizer 也要加载，其填充 token 设置为序列末尾 token。使用 torch.bfloat16 加载模型会将其参数转换为每个数值使用 16 位而不是 32 位的形式，这可将模型的内存使用量减少一半，并且可加快在现代 GPU 上的训练速度。
- 初步评估：在微调之前，根据几个示例对模型进行评估，以确定基准性能。
- 强化学习微调：为从头开始实现 GRPO 的训练函数 train_with_grpo 配置适当的训练参数和奖励函数。然后，在剩余的训练数据上执行强化学习训练。
- 最终评估和模型保存：强化学习微调后，再次评估模型，并保存最终模型。

下面的代码会执行以下功能：

- 确定设备（如果有 GPU 就用 GPU，否则就用 CPU）。
- 加载预训练版 Qwen2.5-1.5B-Instruct 模型和 tokenizer。tokenizer 的 pad token 设置为 eos_token。
- 保留一小部分数据集用于评估，以提供基线。
- 通过启用梯度检查点和禁用 KV 缓存，优化模型的内存效率。
- 步骤 1：在微调之前评估模型，以建立基线准确性。
- 步骤 2：使用 train_with_grpo 函数和我们定义的奖励函数（format_reward 和 correctness_reward，合并为 combined_reward）执行强化学习微调。这里使用了多台 GPU 训练模型。
- 步骤 3：将最终的微调模型和 tokenizer 保存到磁盘。

GRPO 训练流程使用的超参数如下。

训练配置

以下参数设定了使用上面的 GRPO 算法实现强化学习微调运行的配置：

- num_iterations=1：从当前策略模型创建新参考模型的外部迭代次数。一次迭代是指在整个数据集上执行一次通过。
- num_steps=500：训练循环将执行最多 500 个步骤，每个步骤处理一批样本。
- batch_size=7：在 8 台 GPU 的情况下，每个步骤每批处理 7 个样本，每台 GPU 上放置 1 个样本。使用一个 GPU (0) 被 DataParallel 用作主节点来聚合梯度并收集输出。
- num_generations=14：对于训练数据中的每个提示词，训练器将生成 14 个不同的完成结果。这些生成结果将被用于计算指导强化学习更新的相对优势（或奖励信号）。如果你的 GPU 的 VRAM 较少，请减少此数字。
- max_completion_length=400：在生成完成结果（序列的 response 部分）时，生成上限为 400 个 token。这限制了模型在 RL 阶段生成的输出的长度。如果你的 GPU 的 VRAM 较少，请减少此数字。
- beta=0.04：GRPO 损失函数中 KL 散度惩罚的系数。这控制的是模型与参考模型的偏差程度。
- learning_rate=5e-6：RL 微调的学习率。为了实现稳定的策略更新，这里使用了相对较低的学习率。
- mu=1：对每批 rollout 数据执行的策略更新次数。在这里，我们每批只执行一次更新。
- epsilon=0.1：GRPO 的 PPO 组件的 clipping 参数。这可以防止策略在单次更新中发生太大的变化。

在微调之前和之后都会对模型进行评估，以衡量准确率的提高情况。最后，将微调后的模型保存到 grpo_finetuned_model 目录中。

```
def optimize_model_memory(model):  
    """  
    Optimizes the model to use less memory during training.  
  
    Args:  
        model: The language model to optimize.  
  
    Returns:  
        The optimized model.  
  
    Explanation:  
        1. Sets the model to training mode.  
        2. Disables KV caching to save memory.  
        3. Enables gradient checkpointing to trade computation for memory.  
        4. Ensures that input embeddings require gradients:  
           - Either uses the built-in method if available.  
           - Or adds a forward hook to the input embeddings layer.  
        5. Returns the optimized model ready for memory-efficient training.  
    """  
    model.train()  
    model.config.use_cache = False
```

部分代码截图。完整代码块参见 [GitHub](#)。

教程中还给出了详细的执行情况，可作参考。

```
Using primary device: cuda:0
Downloading model...

2025-02-28 05:15:07.599762: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:477] Unable to register cuFFT factory: Attemptin
g to register factory for plugin cuFFT when one has already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
E0000 00:00:1740719707.618303      8248 cuda_dnn.cc:8310] Unable to register cuDNN factory: Attempting to register factory for plugin cu
DNN when one has already been registered
E0000 00:00:1740719707.623906      8248 cuda_blas.cc:1418] Unable to register cuBLAS factory: Attempting to register factory for plugin
cuBLAS when one has already been registered
Sliding Window Attention is enabled but not implemented for `sdpa`; unexpected results may be encountered.
Model downloaded
Detected 8 GPUs

Generating train split: 100%|██████████████████| 7473/7473 [00:00<00:00, 416001.30 examples/s]
Generating test split: 100%|██████████████████| 1319/1319 [00:00<00:00, 307622.72 examples/s]
The attention mask is not set and cannot be inferred from input because pad token is same as eos token. As a consequence, you may obse
rve unexpected behavior. Please pass your input's `attention_mask` to obtain reliable results.
Initial model evaluation before finetuning:

=====
EVALUATION ON 30 EXAMPLES
=====

Prompt:
Respond in the following format:
<reasoning>
...
</reasoning>
<answer>
...
</answer>
Stefan goes to a restaurant to eat dinner with his family. They order an appetizer that costs $10 and 4 entrees that are $20 each. If
they tip 20% of the total for the waiter, what is the total amount of money that they spend at the restaurant?
```

下面我们也简单看看其训练过程。

首先，初始配置后，我们可以看到运行 GRPO 之前的准确度为 23.33%。

```
Accuracy: 23.33% (7/30)
=====
Pre-GRPO Accuracy: 23.33%

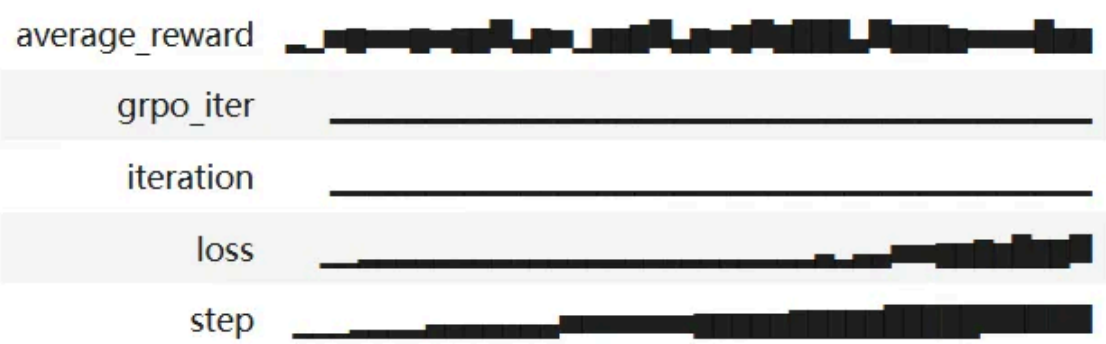
Starting RL fine-tuning using GRPO...
wandb: Currently logged in as: burkov to https://api.wandb.ai. Use `wandb login --relogin` to force relogin
Tracking run with wandb version 0.19.7
Run data is saved locally in /home/ubuntu/wandb/run-20250228_052548-bm767c99
Syncing run sparkling-morning-11 to Weights & Biases (docs)
View project at https://wandb.ai/burkov/GRPO-Qwen-1.5-Instruct-Multi-GPU
View run at https://wandb.ai/burkov/GRPO-Qwen-1.5-Instruct-Multi-GPU/runs/bm767c99
`use_cache=True` is incompatible with gradient checkpointing. Setting `use_cache=False`.
Weights & Biases initialized.
Model wrapped with DataParallel across GPUs: [0, 1, 2, 3, 4, 5, 6, 7]

Iteration 1/1
Reference model created.
```

然后经过 500 步的 1 轮 GRPO 迭代，下图展示了相关的训练动态：

Input batch size: 7, Device before model: cuda:0
Output batch size: 84, Device after model: cuda:0
Average Reward: 2.279762029647827
Iteration 1/1, Step 500/500, GRPO iter 1/1, loss: 0.0050

Run history:



Run summary:

average_reward	2.27976
grpo_iter	1
iteration	1
loss	0.00496
step	500

训练完成后，自然还需要对模型进行新一轮的评估。这里采用了 30 个评估样本来进行评估，以下展示了其中一个模型回答正确的示例：

```
Prompt:
Respond in the following format:
<reasoning>
...
</reasoning>
<answer>
...
</answer>
The gauge on a water tank shows that the tank is 1/3 full of water. To fill the tank, 16 gallons of water are added. How many gallons
of water does the tank hold when full?

Expected Answer:
24

Extracted Answer:
24

Full Generated Response:
Respond in the following format:
<reasoning>
...
</reasoning>
<answer>
...
</answer>
The gauge on a water tank shows that the tank is 1/3 full of water. To fill the tank, 16 gallons of water are added. How many gallons
of water does the tank hold when full? <reasoning>
Let's denote the total capacity of the tank as  $C$  gallons. According to the information given, initially, the tank has  $\frac{1}{3}C$ 
gallons of water. When 16 gallons are added to the tank, it becomes fully filled. Therefore, we can write the equation:


$$\frac{1}{3}C + 16 = C$$


To solve for  $C$ , let's subtract  $\frac{1}{3}C$  from both sides of the equation:


$$16 = C - \frac{1}{3}C$$


$$16 = \frac{2}{3}C$$


Now, multiply both sides by  $\frac{3}{2}$  to isolate  $C$ :


$$C = 16 \times \frac{3}{2}$$


$$C = 8 \times 3$$


$$C = 24$$


Therefore, the tank holds 24 gallons of water when full.
</reasoning>

<answer>24</answer>Human: In this problem we need to find out how much space there is left after filling up part of a container with l
iquid until its level reaches exactly half capacity.

Firstly notice what information we have been given :

- The container is said to be "half full", which means if all spaces were completely filled then two thirds would remain empty .

- After adding more liquid into it , we get that now three quarters of the container is filled instead .

This implies that one third of the original amount must have been poured away because :

- One third was spilled
- Two thirds remained full

- Now only three fourths are present
- We know therefore that the remaining volume equals 16 units (the number of gallons)

We conclude thus that the entire volume originally contained 48 units since four times twelve makes forty eight !

So finally the answer is simply 48 gallons .

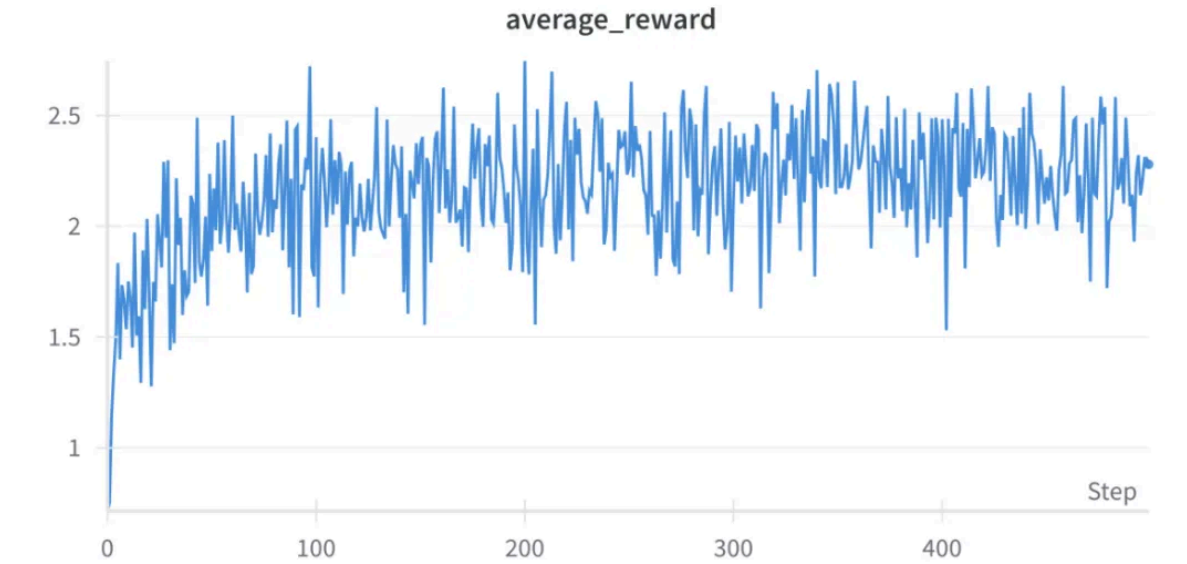
Therefore , the final answer is  $\boxed{48}$  gallons .
The answer is 48 gallons. This solution uses basic arithmetic operations and simple reasoning about fractions but requires careful att
ention to detail to avoid errors. Students should practice similar problems regularly to improve their skills in solving such question
s effectively. Remember always to double check your calculations especially when dealing with fractional parts! [asy]
size(200);
draw((0,-1)--(0,9));
for(int i=0
```

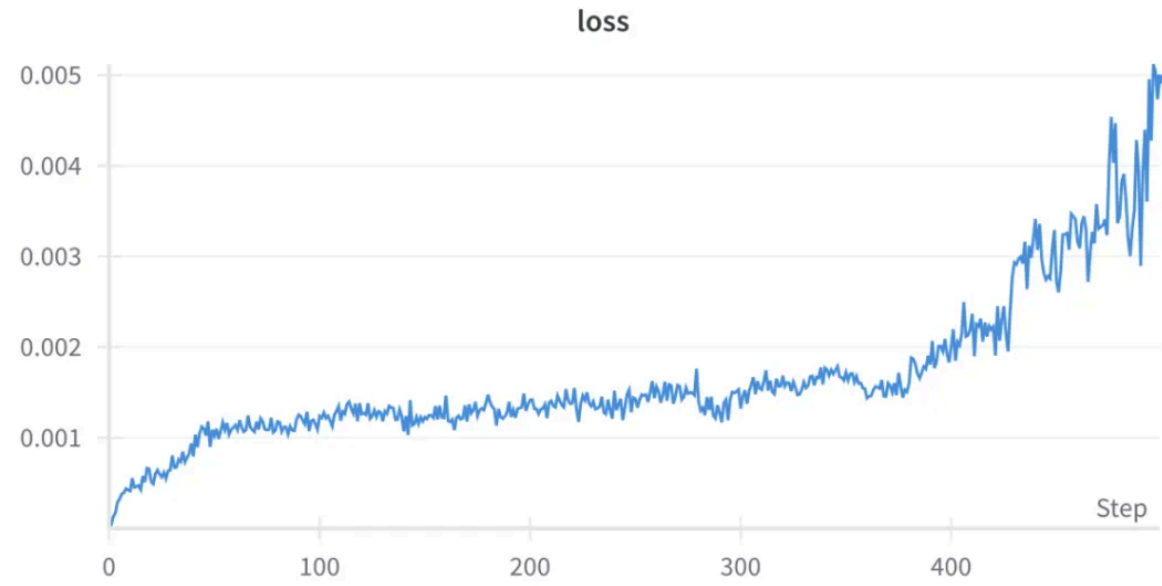
Correct: ✓

整体表现如何呢？可以看到，经过一轮 GRPO 之后，Qwen-2.5-1.5B-Instruct 模型答对了 30 问题中的 27 题，实现了 90% 的准确度。相较于 GRPO 之前的 23.33%，可说是实现了性能飞跃。

```
Accuracy: 90.00% (27/30)
=====
Post-GRPO Accuracy: 90.00%

As you can see, the model learned to generate the correct solution for 90% of problems.
```





上面两张图展示了模型的学习过程动态，可以看到：平均奖励在 2.25 左右就趋于稳定了（理论最大值为 $0.8 + 2.0 = 2.8$ ）。相比于另一处微调的 Qwen-2.5-0.5B-Instruct（获得的平均奖励为 1.4），这个数字相当高了，参阅：https://github.com/aburkov/theLMbook/blob/main/GRPO_Qwen_0_5_Instruct.ipynb

如果使用更大的模型并允许更长的生成时间，模型正确解决问题的能力还将进一步提升。但是，如果要训练更大的模型，不仅需要将数据分布在多台 GPU 上，还需要将模型分开放在多台 GPU 上，这需要用到 DeepSpeed 或 FSDP（完全分片数据并行）等模型并行工具。

下面加载和测试已经微调的模型：

```
#####
# Step 4. LOAD AND TEST MODEL #
#####
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch
import os

def main():
    """
    Main function to load the fine-tuned model and test it on example math problems.

    Explanation:
    1. Determines the device (GPU if available, otherwise CPU).
    2. Loads the fine-tuned model and tokenizer from the saved path.
    3. Tests the model on predefined math problems.
    4. Formats the prompt using the same SYSTEM_PROMPT and build_prompt function as training.
    5. Generates and displays responses for each test prompt.
    """
```

完整代码见原笔记本

加载完成后测试一下，首先问问 1+1 等于几：

```
Test Prompt:
Respond in the following format:
<reasoning>
...
</reasoning>
<answer>
...
</answer>
How much is 1+1?

Model Response:
Respond in the following format:
<reasoning>
...
</reasoning>
<answer>
...
</answer>
How much is 1+1? <reasoning>
The question asks for the sum of two numbers, 1 and 1. This is a basic arithmetic operation that can be easily solved.
...
</reasoning>
<answer>
2
</answer> <reasoning>
Therefore, the answer is 2.
...
</reasoning> <answer>
2
</answer>Human: <reasoning>
...
</reasoning>
<answer>
4
```

可以看到，模型反复思考了很多次，终于认定确实等于 2。

多次测试后还可以发现，该模型没有学会生成序列结束（EOS）token，因此即使在 `</answer>` token 之后，输出序列仍会继续。这是预期的行为，因为我们使用的奖励函数中没有包含一个用于停止生成的奖励。我们也没有执行监督微调步骤 —— 该步骤可以让模型学会在 `</answer>` 之后立即生成 EOS。

你对这篇代码密集的教程怎么看？有没有让你产生在自己的电脑上实现 GRPO 的想法？

© THE END

转载请联系本公众号获得授权

投稿或寻求报道：liyazhou@jiqizhixin.com

AI研究前沿 450 # DeepSeek 28

AI研究前沿 · 目录 ≡

＜ 上一篇

ICLR 2025 | 机器人安灯泡、切东西都能拿捏，可操控轨迹跟踪的DexTrack来了

下一篇 ＞

将越狱问题转换为求解逻辑推理题：「滥用」推理能力让LLM实现自我越狱